The Journal of Open Source Software

NIELIT StudentHub: A Zero-Opex, Serverless Institutional Knowledge Management System

Lovnish Verma [1*] **and Sarwan Singh** [1*]

1 National Institute of Electronics & Information Technology (NIELIT), Ropar, India * These authors contributed equally.

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor:

Submitted: 24 January 2026
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

DRAFT

Summary

In the domain of engineering education, the preservation and social dissemination of student projects remain a significant challenge. While global platforms like LinkedIn and GitHub provide visibility, they lack the institutional context and data sovereignty required for internal academic assessment. Conversely, deploying dedicated institutional repositories typically incurs significant recurring cloud costs (OpEx) and maintenance overhead, leading to "digital rot" upon funding cessation (Vare, 2021).

`NIELIT StudentHub` is a hyper-localized professional social network built on a novel **Zero-Opex Architecture**. By orchestrating commodity SaaS tools—specifically repurposing Google Sheets as a relational persistence layer, Google Apps Script as a serverless API gateway, and GitHub Pages for static hosting—it provides enterprise-grade features (secure authentication, engagement algorithms, and media optimization) with **$0.00 annual maintenance costs**. The software introduces a proprietary "Time-Decay Engagement Heuristic" and a "Self-Healing Cryptographic Migration" protocol, proving that robust software engineering principles can be applied to low-code infrastructure to democratize digital visibility for resource-constrained institutions.

Statement of need

The modern engineering curriculum emphasizes project-based learning (PBL). However, a significant gap exists between the creation of academic projects and their long-term archival. Most student projects remain trapped in local storage, ephemeral flash drives, or are uploaded to disparate personal repositories, rendering them inaccessible to future cohorts.

The primary barrier to deploying dynamic, social-driven institutional platforms in developing nations and resource-constrained institutions is **infrastructure cost**. A standard MERN (MongoDB, Express, React, Node.js) stack requires persistent server hosting (e.g., AWS EC2), managed database services (e.g., AWS RDS), and regular security patching. For government and educational institutions with limited recurring budgets, this operational expense creates a sustainability crisis, often leading to platform abandonment once initial grant funding expires (Heeks & Molla, 2009).

`NIELIT StudentHub` bridges the gap between "No-Code" tools (which often lock users into subscriptions) and "Pro-Code" architectures (which require expensive servers). It allows institutions to deploy a fully functional, self-healing, and secure project repository using only free-tier ecosystems. It is designed to be used by university IT departments, student developers, and educational administrators who need to maintain institutional memory without incurring infrastructure debt (Syeda et al., 2025).

State of the field

Existing solutions generally fall into two categories: commercial professional networks (e.g., LinkedIn, Behance) and enterprise Learning Management Systems (e.g., Blackboard, Canvas). Commercial networks suffer from a low signal-to-noise ratio for academic work and strip the institution of data ownership. Enterprise LMS solutions are prohibitively expensive for small government institutions or individual departments and often lack social engagement features like peer-review comments or "trending" feeds.

While "No-Code" tools like Glide Apps allow for rapid application development using spreadsheets, they impose subscription fees for advanced features such as white-labeling or high row counts (Glide Apps, 2024). Conversely, Static Site Generators (SSGs) like Jekyll are free but lack dynamic write capabilities. `NIELIT StudentHub` represents a "Frugal Information System" (Watson et al., 2013) that utilizes a "Pro-Code" methodology on "No-Code" infrastructure, writing custom JavaScript to force commodity tools to behave like enterprise servers.

Software architecture

`NIELIT StudentHub` utilizes a **Serverless Micro-Service Architecture** (Roberts, 2016) that decouples the frontend from the data layer, ensuring high availability without dedicated servers.

1. Frontend: The Single Page Application (SPA)

The client-side is built using vanilla HTML5/CSS3/JavaScript, hosted on GitHub Pages. This ensures global content delivery via a Content Delivery Network (CDN) at zero cost. To overcome the stateless nature of the backend, the system implements a hybrid session model. User session tokens are stored in the browser's `localStorage` (specifically the `NIELIT StudentHub_user` key) and are transmitted in the header of every API request for server-side validation.

2. Middleware: The Serverless Gateway

Google Apps Script (GAS) serves as the backend logic layer. It acts as an API Gateway, translating HTTP requests into Google Sheets operations. * **Endpoint Management:** The system utilizes the `doGet(e)` and `doPost(e)` triggers to handle RESTful API requests. The `e.parameter` object is parsed to route requests to specific controllers (e.g., `handleLogin`, `handleGetProjects`). * **Sanitization:** All inputs undergo strict HTML entity encoding on the server side to prevent Cross-Site Scripting (XSS) attacks before storage.

3. Persistence Layer: Hyper-Converged Data Store

Instead of a traditional SQL database, the system utilizes **Google Sheets** as a NoSQL-like store. The database is normalized into **six** distinct sheets to maintain data integrity and separate user authentication from social interactions:

- **Core Entities:**
  - `Users`: Stores authentication credentials (UserID, Email, PasswordHash, Salt).
  - `Profiles`: Stores public user metadata (Bio, Role, Social Links) to keep the auth table lightweight.
  - `Projects`: Stores ProjectID, Title, Description, TechStack, and Cloudinary MediaURLs.
- **Relational Tables:**
  - `Comments`: Links ProjectID and UserID for textual feedback.
  - `Upvotes`: Tracks UserID-ProjectID pairs to prevent duplicate project voting.
  - `ProfileLikes`: Tracks peer-to-peer acknowledgement on user profiles.

DRAFT

Key algorithms

The research novelty lies in the algorithmic adaptation of limited tools to simulate enterprise features.

Time-Decay Engagement Heuristic

To ensure the "Trending" feed remains dynamic, `NIELIT StudentHub` rejects simple summation (e.g., "Most Likes"), which would allow early projects to accumulate insurmountable leads. Instead, it implements a **Gravity Decay Algorithm** similar to those used by news aggregators. The ranking score S for a project p is calculated as:

$$S_p = \frac{(W_u \cdot U_p) + (W_c \cdot C_p)}{\sqrt{\Delta t + 1}}$$

Where: * U_p is the count of Upvotes. * C_p is the count of Comments. * W_u (Weight of Upvote) $= 2$. * W_c (Weight of Comment) $= 3$. * Δt is the age of the project in days since upload.

This $O(N)$ operation ensures that a new project with high engagement ranks higher than a year-old project with accumulated engagement, fostering a "Freshness First" ecosystem.

Server-Side Pagination and Slicing

Google Apps Script imposes a strict 6-minute execution limit (Google Developers, 2025). To prevent timeout errors as the dataset grows, `NIELIT StudentHub` implements strict Server-Side Pagination. The API accepts page and `limit` parameters. The optimization `projects.slice(startIndex, endIndex)` is performed *after* filtering but *before* JSON serialization. This reduces the payload size by approximately 95% compared to fetching the full dataset, ensuring response times remain under 1.5 seconds even as the database grows.

Self-Healing Cryptographic Migration

The system implements an "Opportunistic Security Upgrade" protocol to manage technical debt. It handles the transition from legacy hash formats to modern security standards without forcing a global password reset. During login, the system checks if the stored hash matches the legacy format. If verified, it automatically generates a salt, re-hashes the password using SHA-256, and updates the database record silently.

```
if (storedHash === oldHash) {
    // Authenticate User
    const newSalt = generateSalt();
    const newHash = hash(password + newSalt);
    updateUserRecord(email, newHash, newSalt);
}
```

Performance and scalability

Critics of spreadsheet-based backends often cite scalability as a fatal flaw. However, our analysis proves that for specific institutional scales ($N < 5,000$ users), the architecture is mathematically sound.

We analyzed the system against Google's published quotas (Google Developers, 2025):

- **Concurrency:** Google Apps Script limits simultaneous executions to approximately 30. For a campus of 2,000 students, assuming a Peak Concurrency Factor of 5% (100 users

DRAFT

active in a single hour), the average requests per second (RPS) remain below the throttle threshold.

- **Data Volume:** A Google Sheet can hold 10 million cells. With an average row utilizing 20 columns, the system can store 500,000 projects. At a creation rate of 10 projects/day, the database has a theoretical lifespan of **27 years** before archival is needed.
- **Latency:** While read/write speeds (~800ms - 1.2s) are slower than traditional SQL (~20ms), this latency is acceptable for a non-real-time academic archiving application where financial sustainability is the priority.

AI usage disclosure

No generative AI tools were used in the development of this software. AI assistance was utilized for the formatting and structuring of this manuscript.

Acknowledgements

We acknowledge the support of the National Institute of Electronics & Information Technology (NIELIT), Ropar, for providing the testing environment for this research.

References

Glide Apps. (2024). *Glide pricing and features*. https://www.glideapps.com/pricing

Google Developers. (2025). *Quotas for google services*. https://developers.google.com/apps-script/guides/services/quotas

Heeks, R., & Molla, A. (2009). Impact assessment of ICT-for-development projects: A compendium of approaches. *Development Informatics Working Paper*, *36*. http://dx.doi.org/10.2139/ssrn.3477380

Roberts, M. (2016). Serverless architectures. *MartinFowler.com*. https://martinfowler.com/articles/serverless.html

Syeda, N., Shah, H., Singh, R., Jaju, S., Kumar, S., Chhabra, G., & Spichkova, M. (2025). Analysis of cost-efficiency of serverless approaches. *arXiv Preprint arXiv:2506.05836*. https://doi.org/10.48550/arXiv.2506.05836

Vare, P. (2021). Exploring the impacts of student-led sustainability projects with secondary school students and teachers. *Sustainability*, *13*(5), 2790. https://doi.org/10.3390/su13052790

Watson, R. T., Kunene, K. N., & Islam, M. S. (2013). Frugal information systems (IS). *Information Technology for Development*, *19*(2), 176–187. https://doi.org/10.1080/02681102.2012.714349

DRAFT